GEREKLİ KÜTÜPHANELERİN YÜKLENMESİ

In [2]:
```
import numpy as np
import os
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.applications import VGG16
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import ReduceLROnPlateau
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[2], line 1
----> 1 import numpy as np
      2 import os
      3 import pandas as pd

ModuleNotFoundError: No module named 'numpy'
```

Veri Setini Yükleme

In [ ]:
```
# Veri yolu
import os
# os.getcwd() -> notebook hangi klasorden acilirsa orasi baz alinir
base_dir  = os.path.join(os.getcwd(), "Snake Images")
train_dir = os.path.join(base_dir, "train")
test_dir  = os.path.join(base_dir, "test")
print("Train:", train_dir)
print("Test: ", test_dir)
```

Model Parametreleri ve Eğitim Ayarları

In [ ]:
```
# Model parametreleri
input_shape = (224, 224, 3)  # Görüntü boyutları
num_classes = 1  # Binary sınıflandırma (Venomous/Non-Venomous)
batch_size = 32
```

Veri Artırma ve Veri Yükleme (Data Augmentation & Generators)

In [ ]:
```
# Eğitim için veri artırma (Data Augmentation)
train_datagen = ImageDataGenerator(
    rescale=1.0/255,                # Normalizasyon [0, 1]
    rotation_range=20,             # Görüntü döndürme (derece cinsinden)
    width_shift_range=0.2,         # Görüntü genişliğini kaydırma
    height_shift_range=0.2,        # Görüntü yüksekliğini kaydırma
    shear_range=0.2,               # Kesme işlemi
    zoom_range=0.2,                # Yakınlaştırma
    horizontal_flip=True,          # Yatay çevirme
    fill_mode='nearest'            # Doldurma modu (boşlukları doldur)
)

test_datagen = ImageDataGenerator(rescale=1.0/255)  # Sadece normalize

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224),
    batch_size=batch_size,
    class_mode='binary',  # Binary sınıflandırma için
    classes=['Non Venomous', 'Venomous']  # Klasörleri manuel olarak belirtin
)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(224, 224),
    batch_size=batch_size,
    class_mode='binary',  # Binary sınıflandırma için
    classes=['Non Venomous', 'Venomous']  # Klasörleri manuel olarak belirtin
)
```

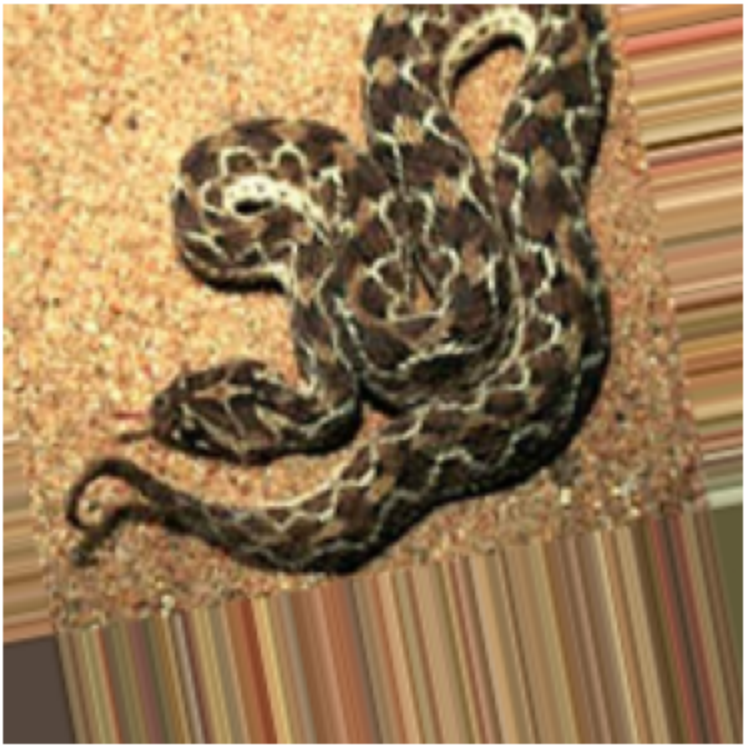
Eğitim ve Test Verisi Kontrolleri

In [ ]:
```
# Eğitim etiketlerini kontrol et
print("Eğitim Etiketleri:")
print(np.unique(train_generator.classes))

# Doğrulama etiketlerini kontrol et
print("Doğrulama Etiketleri:")
print(np.unique(test_generator.classes))

# Eğitim setindeki sınıf dağılımı
unique, counts = np.unique(train_generator.classes, return_counts=True)
print("Eğitim Sınıf Dağılımı:")
print(dict(zip(unique, counts)))

# Doğrulama setindeki sınıf dağılımı
unique, counts = np.unique(test_generator.classes, return_counts=True)
print("Doğrulama Sınıf Dağılımı:")
print(dict(zip(unique, counts)))

print("Sınıf İsimleri (Eğitim):")
print(train_generator.class_indices)
```

```
Eğitim Etiketleri:
[0 1]
Doğrulama Etiketleri:
[0 1]
Eğitim Sınıf Dağılımı:
{0: 715, 1: 1060}
Doğrulama Sınıf Dağılımı:
{0: 128, 1: 141}
Sınıf İsimleri (Eğitim):
{'Non_Venomous': 0, 'Venomous': 1}
```

Etiket: 0.0

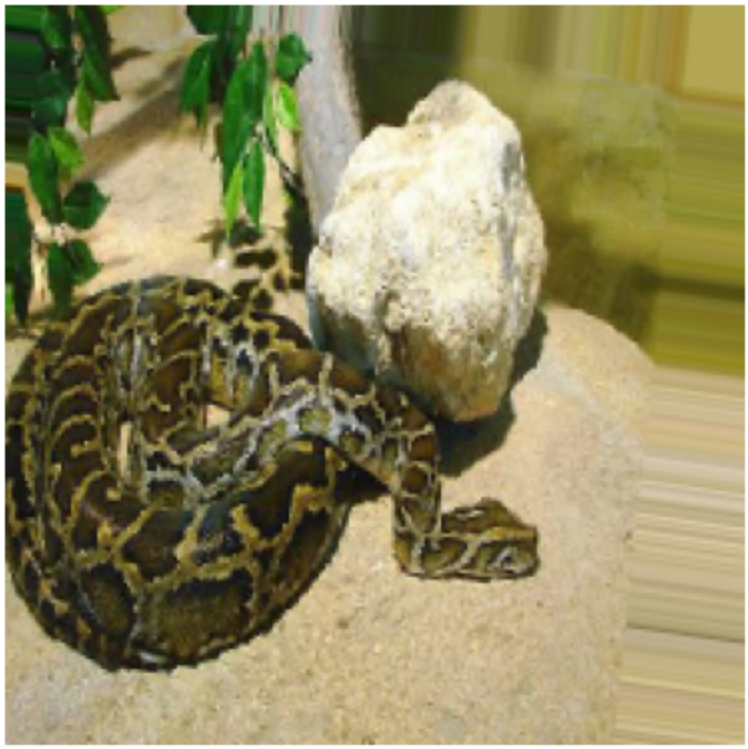
Etiket: 1.0



Etiket: 0.0

Etiket: 1.0

Etiket: 0.0



Veri Görselleştirme - Eğitim Verisi Örnekleri

In [ ]:
```
# Eğitim setinden birkaç örnek al
x_batch, y_batch = next(train_generator)
```

```
# İlk 5 görüntüyü ve etiketlerini göster
for i in range(5):
    plt.imshow(x_batch[i])
    plt.title(f"Etiket: {y_batch[i]}")
    plt.axis('off')
    plt.show()
```

Model Oluşturma - ResNet50 Transfer Learning

In [ ]:
```
from tensorflow.keras.applications import ResNet50
# ResNet50 Modeli Oluşturma
def create_resnet50_model(input_shape, num_classes):
    base_model = ResNet50(weights='imagenet', include_top=False, input_shape=input_

    # Taban katmanları dondur
    for layer in base_model.layers:
        layer.trainable = False

    # Yeni katmanlar ekle
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(128, activation='relu')(x)
    x = Dropout(0.5)(x)
    predictions = Dense(1, activation='sigmoid')(x)

    model = Model(inputs=base_model.input, outputs=predictions)
    return model

# Modeli oluştur
resnet_model = create_resnet50_model(input_shape=input_shape, num_classes=num_class
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/r
esnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/50
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/trainers/data_adapters/py_dataset_
adapter.py:122: UserWarning: Your `PyDataset` class should call `super().__init__(**
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing
`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore
d.
  self._warn_if_super_not_called()
```

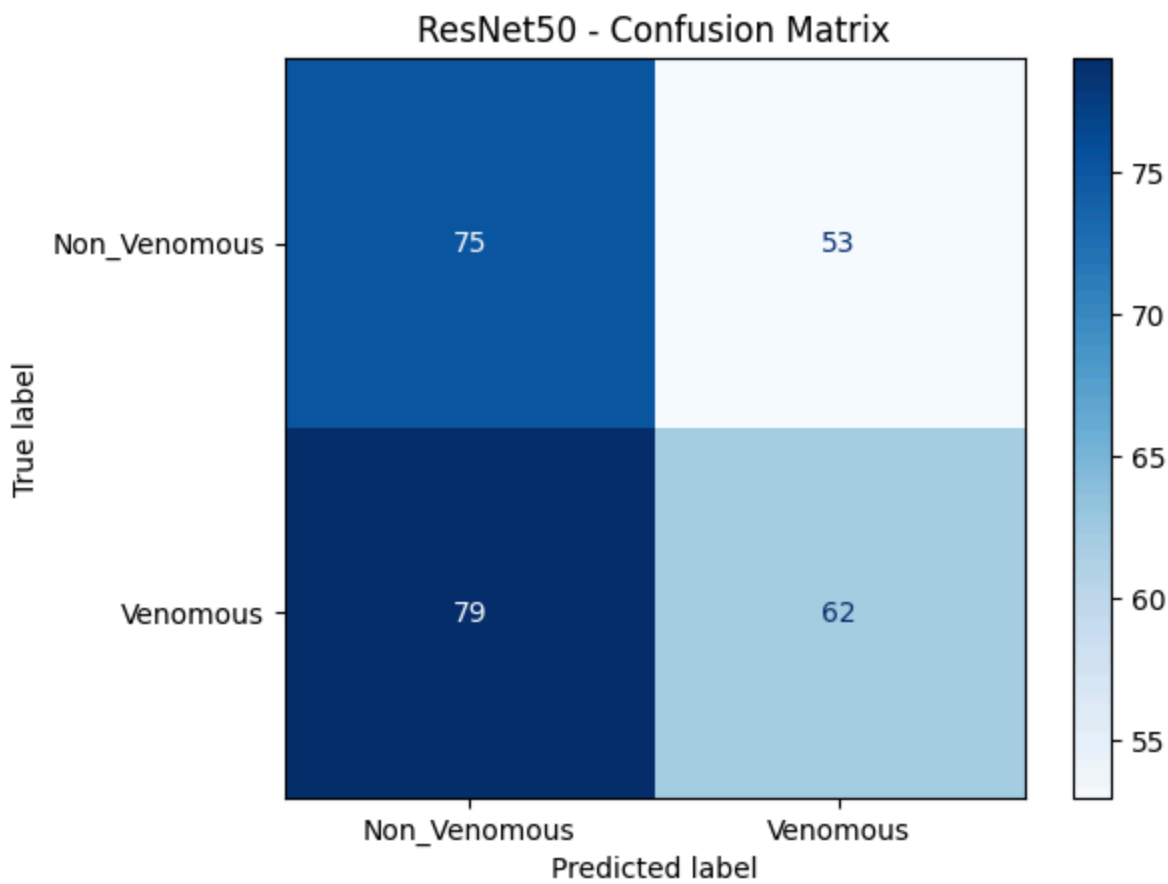
```
56/56 ━━━━━━━━━━━━━━━━━━━━ 70s 852ms/step - accuracy: 0.5132 - loss: 0.7967 - val_ac
curacy: 0.4647 - val_loss: 0.6962 - learning_rate: 1.0000e-04
Epoch 2/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 32s 517ms/step - accuracy: 0.5302 - loss: 0.7333 - val_ac
curacy: 0.5130 - val_loss: 0.6935 - learning_rate: 1.0000e-04
Epoch 3/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 32s 505ms/step - accuracy: 0.5103 - loss: 0.7110 - val_ac
curacy: 0.5093 - val_loss: 0.6961 - learning_rate: 1.0000e-04
Epoch 4/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 30s 480ms/step - accuracy: 0.4683 - loss: 0.7101 - val_ac
curacy: 0.4907 - val_loss: 0.6911 - learning_rate: 1.0000e-04
Epoch 5/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 31s 499ms/step - accuracy: 0.4896 - loss: 0.7002 - val_ac
curacy: 0.5167 - val_loss: 0.6875 - learning_rate: 1.0000e-04
Epoch 6/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 504ms/step - accuracy: 0.5077 - loss: 0.6982 - val_ac
curacy: 0.5242 - val_loss: 0.6874 - learning_rate: 1.0000e-04
Epoch 7/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 31s 495ms/step - accuracy: 0.5303 - loss: 0.6928 - val_ac
curacy: 0.5242 - val_loss: 0.6860 - learning_rate: 1.0000e-04
Epoch 8/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 39s 481ms/step - accuracy: 0.5731 - loss: 0.6850 - val_ac
curacy: 0.5428 - val_loss: 0.6887 - learning_rate: 1.0000e-04
Epoch 9/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 44s 513ms/step - accuracy: 0.4892 - loss: 0.6879 - val_ac
curacy: 0.5725 - val_loss: 0.6850 - learning_rate: 1.0000e-04
Epoch 10/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 32s 512ms/step - accuracy: 0.5125 - loss: 0.6874 - val_ac
curacy: 0.5279 - val_loss: 0.6862 - learning_rate: 1.0000e-04
Epoch 11/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 39s 475ms/step - accuracy: 0.5317 - loss: 0.6839 - val_ac
curacy: 0.5204 - val_loss: 0.6862 - learning_rate: 1.0000e-04
Epoch 12/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 29s 447ms/step - accuracy: 0.5196 - loss: 0.6897 - val_ac
curacy: 0.5279 - val_loss: 0.6857 - learning_rate: 1.0000e-04
Epoch 13/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 36s 370ms/step - accuracy: 0.4948 - loss: 0.6916 - val_ac
curacy: 0.5465 - val_loss: 0.6849 - learning_rate: 5.0000e-05
Epoch 14/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 372ms/step - accuracy: 0.5511 - loss: 0.6821 - val_ac
curacy: 0.5390 - val_loss: 0.6846 - learning_rate: 5.0000e-05
Epoch 15/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 382ms/step - accuracy: 0.5232 - loss: 0.6861 - val_ac
curacy: 0.5576 - val_loss: 0.6824 - learning_rate: 5.0000e-05
Epoch 16/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 367ms/step - accuracy: 0.5168 - loss: 0.6914 - val_ac
curacy: 0.5465 - val_loss: 0.6834 - learning_rate: 5.0000e-05
Epoch 17/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 42s 391ms/step - accuracy: 0.5456 - loss: 0.6884 - val_ac
curacy: 0.5428 - val_loss: 0.6831 - learning_rate: 5.0000e-05
Epoch 18/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 390ms/step - accuracy: 0.5090 - loss: 0.6942 - val_ac
curacy: 0.5428 - val_loss: 0.6824 - learning_rate: 5.0000e-05
Epoch 19/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 25s 396ms/step - accuracy: 0.5554 - loss: 0.6845 - val_ac
curacy: 0.5465 - val_loss: 0.6820 - learning_rate: 2.5000e-05
```

```
Epoch 20/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 397ms/step - accuracy: 0.5342 - loss: 0.6858 - val_ac
curacy: 0.5651 - val_loss: 0.6818 - learning_rate: 2.5000e-05
Epoch 21/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 391ms/step - accuracy: 0.5177 - loss: 0.6814 - val_ac
curacy: 0.5576 - val_loss: 0.6820 - learning_rate: 2.5000e-05
Epoch 22/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 386ms/step - accuracy: 0.5067 - loss: 0.6857 - val_ac
curacy: 0.5576 - val_loss: 0.6820 - learning_rate: 2.5000e-05
Epoch 23/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 389ms/step - accuracy: 0.5324 - loss: 0.6877 - val_ac
curacy: 0.5688 - val_loss: 0.6815 - learning_rate: 2.5000e-05
Epoch 24/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 390ms/step - accuracy: 0.5165 - loss: 0.6837 - val_ac
curacy: 0.5651 - val_loss: 0.6815 - learning_rate: 2.5000e-05
Epoch 25/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 378ms/step - accuracy: 0.5234 - loss: 0.6933 - val_ac
curacy: 0.5613 - val_loss: 0.6809 - learning_rate: 2.5000e-05
Epoch 26/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 42s 390ms/step - accuracy: 0.4863 - loss: 0.6944 - val_ac
curacy: 0.5465 - val_loss: 0.6812 - learning_rate: 2.5000e-05
Epoch 27/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 383ms/step - accuracy: 0.4939 - loss: 0.6906 - val_ac
curacy: 0.5651 - val_loss: 0.6807 - learning_rate: 2.5000e-05
Epoch 28/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 366ms/step - accuracy: 0.5219 - loss: 0.6829 - val_ac
curacy: 0.5688 - val_loss: 0.6808 - learning_rate: 2.5000e-05
Epoch 29/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 27s 376ms/step - accuracy: 0.5525 - loss: 0.6838 - val_ac
curacy: 0.5502 - val_loss: 0.6807 - learning_rate: 2.5000e-05
Epoch 30/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 37s 364ms/step - accuracy: 0.5209 - loss: 0.6843 - val_ac
curacy: 0.5465 - val_loss: 0.6807 - learning_rate: 2.5000e-05
Epoch 31/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 368ms/step - accuracy: 0.5392 - loss: 0.6813 - val_ac
curacy: 0.5613 - val_loss: 0.6805 - learning_rate: 1.2500e-05
Epoch 32/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 383ms/step - accuracy: 0.4844 - loss: 0.6899 - val_ac
curacy: 0.5539 - val_loss: 0.6803 - learning_rate: 1.2500e-05
Epoch 33/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 25s 384ms/step - accuracy: 0.5046 - loss: 0.6911 - val_ac
curacy: 0.5576 - val_loss: 0.6803 - learning_rate: 1.2500e-05
Epoch 34/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 369ms/step - accuracy: 0.5134 - loss: 0.6877 - val_ac
curacy: 0.5613 - val_loss: 0.6804 - learning_rate: 1.2500e-05
Epoch 35/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 375ms/step - accuracy: 0.5257 - loss: 0.6841 - val_ac
curacy: 0.5502 - val_loss: 0.6803 - learning_rate: 1.2500e-05
Epoch 36/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 371ms/step - accuracy: 0.5504 - loss: 0.6835 - val_ac
curacy: 0.5465 - val_loss: 0.6804 - learning_rate: 6.2500e-06
Epoch 37/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 379ms/step - accuracy: 0.5366 - loss: 0.6832 - val_ac
curacy: 0.5576 - val_loss: 0.6803 - learning_rate: 6.2500e-06
Epoch 38/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 25s 395ms/step - accuracy: 0.5200 - loss: 0.6917 - val_ac
```

```
curacy: 0.5725 - val_loss: 0.6802 - learning_rate: 6.2500e-06
Epoch 39/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 379ms/step - accuracy: 0.5131 - loss: 0.6926 - val_ac
curacy: 0.5613 - val_loss: 0.6801 - learning_rate: 3.1250e-06
Epoch 40/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 365ms/step - accuracy: 0.5198 - loss: 0.6778 - val_ac
curacy: 0.5613 - val_loss: 0.6802 - learning_rate: 3.1250e-06
Epoch 41/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 367ms/step - accuracy: 0.5300 - loss: 0.6810 - val_ac
curacy: 0.5613 - val_loss: 0.6802 - learning_rate: 3.1250e-06
Epoch 42/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 25s 395ms/step - accuracy: 0.5143 - loss: 0.6869 - val_ac
curacy: 0.5576 - val_loss: 0.6803 - learning_rate: 3.1250e-06
Epoch 43/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 365ms/step - accuracy: 0.5007 - loss: 0.6888 - val_ac
curacy: 0.5651 - val_loss: 0.6802 - learning_rate: 1.5625e-06
Epoch 44/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 371ms/step - accuracy: 0.5140 - loss: 0.6891 - val_ac
curacy: 0.5688 - val_loss: 0.6802 - learning_rate: 1.5625e-06
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
saving.save_model(model)`. This file format is considered legacy. We recommend using
instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.savin
g.save_model(model, 'my_model.keras')`.
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 1s 79ms/step - accuracy: 0.5541 - loss: 0.6749
Test Loss: 0.6801440119743347, Test Accuracy: 0.5613383054733276
9/9 ━━━━━━━━━━━━━━━━━━━━ 8s 433ms/step
```



Model Derleme ve Eğitim - ResNet50

In [ ]:
```
# Modeli derle
resnet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Callbacks tanımlama
lr_scheduler_res = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    min_lr=1e-7
)
early_stop_res = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

# Sınıf ağırlıkları
class_weights_array = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)
class_weights = {i: weight for i, weight in enumerate(class_weights_array)}

# Modeli eğitme
history_res = resnet_model.fit(
    train_generator,
    epochs=50,
    validation_data=test_generator,
    class_weight=class_weights,
    callbacks=[lr_scheduler_res, early_stop_res]
)
```

Model Kaydetme ve Değerlendirme - ResNet50

In [ ]:
```
# Modeli kaydet
resnet_model.save("resnet50_snake_model.keras")

# Modeli değerlendirme
test_loss_res, test_accuracy_res = resnet_model.evaluate(test_generator)
print(f"Test Loss: {test_loss_res}, Test Accuracy: {test_accuracy_res}")
```

Confusion Matrix - ResNet50

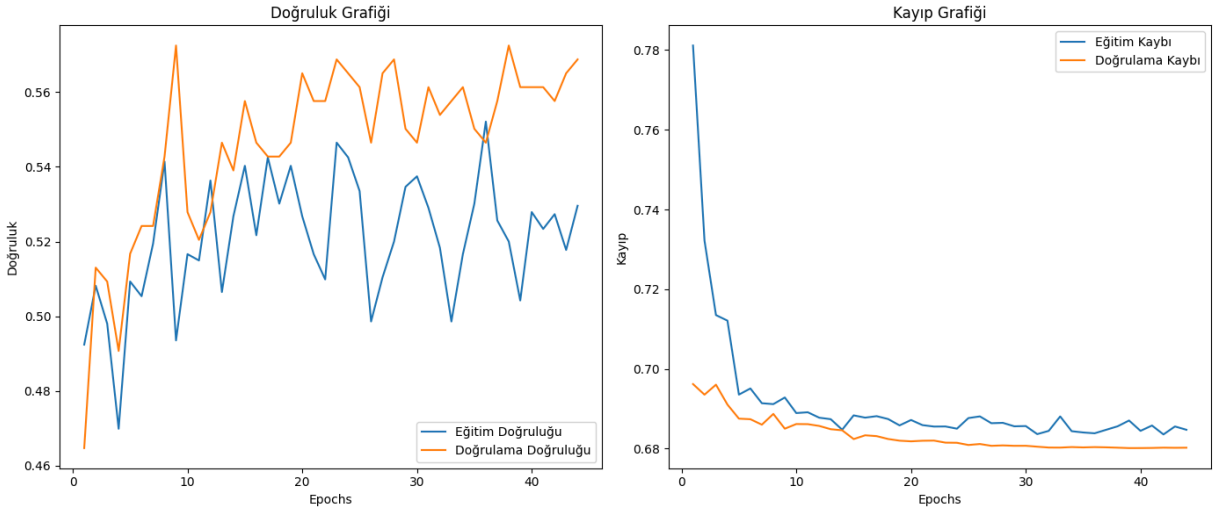
In [ ]:
```
# Confusion Matrix için tahminler
test_labels = test_generator.classes  # Test verisinin gerçek etiketleri
```

```
class_names = list(test_generator.class_indices.keys())  # Sınıf isimleri

resnet_predictions = resnet_model.predict(test_generator)
resnet_predicted_labels = np.round(resnet_predictions).astype(int)  # Tahminler bin

# Confusion Matrix Çizimi
cm = confusion_matrix(test_labels, resnet_predicted_labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(cmap=plt.cm.Blues)
plt.title("ResNet50 - Confusion Matrix")
plt.show()
```

Eğitim Süreci Grafikleri - ResNet50

In [ ]:

```
train_acc_res = history_res.history['accuracy']
val_acc_res = history_res.history['val_accuracy']
train_loss_res = history_res.history['loss']
val_loss_res = history_res.history['val_loss']

epochs_res = range(1, len(train_acc_res) + 1)

plt.figure(figsize=(14, 6))

# Doğruluk grafiği
plt.subplot(1, 2, 1)
plt.plot(epochs_res, train_acc_res, label='Eğitim Doğruluğu')
plt.plot(epochs_res, val_acc_res, label='Doğrulama Doğruluğu')
plt.title('Doğruluk Grafiği')
plt.xlabel('Epochs')
plt.ylabel('Doğruluk')
plt.legend()

# Kayıp grafiği
plt.subplot(1, 2, 2)
plt.plot(epochs_res, train_loss_res, label='Eğitim Kaybı')
plt.plot(epochs_res, val_loss_res, label='Doğrulama Kaybı')
plt.title('Kayıp Grafiği')
plt.xlabel('Epochs')
plt.ylabel('Kayıp')
plt.legend()

plt.tight_layout()
plt.show()
```



Transfer Learning - VGG16

In [ ]:
```
# Transfer Learning Modeli
def create_transfer_learning_model(input_shape, num_classes):
    base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_sha

    # Yeni katmanlar ekleyin
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(128, activation='relu')(x)
    x = Dropout(0.4)(x)
    predictions = Dense(num_classes, activation='sigmoid' if num_classes == 1 else

    model = Model(inputs=base_model.input, outputs=predictions)

    # Önceden eğitilmiş katmanları dondur
    for layer in base_model.layers:
        layer.trainable = False

    return model
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/v
gg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

Model Derleme ve Eğitim - VGG16

In [ ]:
```
# Sınıf ağırlıklarını hesaplama
class_weights_array = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)

# Sınıf ağırlıklarını dict formatına dönüştürme
class_weights = {i: weight for i, weight in enumerate(class_weights_array)}

# ReduceLROnPlateau tanımlama
```

```
lr_scheduler_vgg = ReduceLROnPlateau(
    monitor='val_loss',   # Hangi metriği takip edecek
    factor=0.5,           # Öğrenme hızını yarıya indir
    patience=3,           # Kaç epoch bekleyecek
    min_lr=1e-6           # Minimum öğrenme hızı
)

early_stop_vgg = EarlyStopping(
    monitor='val_loss',
    patience=2,  # Bekleme süresini 10 epoch'a çıkardık
    min_delta=0.001,
    restore_best_weights=True
)
```

```
Epoch 1/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 118s 1s/step - accuracy: 0.5261 - loss: 0.7184 - val_accu
racy: 0.6283 - val_loss: 0.6704 - learning_rate: 1.0000e-05
Epoch 2/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 35s 575ms/step - accuracy: 0.5465 - loss: 0.7046 - val_ac
curacy: 0.6691 - val_loss: 0.6479 - learning_rate: 1.0000e-05
Epoch 3/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 35s 570ms/step - accuracy: 0.5676 - loss: 0.6740 - val_ac
curacy: 0.6580 - val_loss: 0.6193 - learning_rate: 1.0000e-05
Epoch 4/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 35s 569ms/step - accuracy: 0.6269 - loss: 0.6362 - val_ac
curacy: 0.7175 - val_loss: 0.5833 - learning_rate: 1.0000e-05
Epoch 5/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 34s 565ms/step - accuracy: 0.6726 - loss: 0.6064 - val_ac
curacy: 0.6357 - val_loss: 0.6307 - learning_rate: 1.0000e-05
Epoch 6/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 36s 574ms/step - accuracy: 0.6796 - loss: 0.5945 - val_ac
curacy: 0.7398 - val_loss: 0.5439 - learning_rate: 1.0000e-05
Epoch 7/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 570ms/step - accuracy: 0.7046 - loss: 0.5557 - val_ac
curacy: 0.7249 - val_loss: 0.5262 - learning_rate: 1.0000e-05
Epoch 8/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 34s 569ms/step - accuracy: 0.7259 - loss: 0.5476 - val_ac
curacy: 0.7398 - val_loss: 0.5187 - learning_rate: 1.0000e-05
Epoch 9/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 35s 570ms/step - accuracy: 0.7631 - loss: 0.5028 - val_ac
curacy: 0.7361 - val_loss: 0.5120 - learning_rate: 1.0000e-05
Epoch 10/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 36s 573ms/step - accuracy: 0.7491 - loss: 0.5171 - val_ac
curacy: 0.7844 - val_loss: 0.4607 - learning_rate: 1.0000e-05
Epoch 11/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 573ms/step - accuracy: 0.7707 - loss: 0.4730 - val_ac
curacy: 0.7770 - val_loss: 0.4736 - learning_rate: 1.0000e-05
Epoch 12/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 573ms/step - accuracy: 0.7966 - loss: 0.4539 - val_ac
curacy: 0.7844 - val_loss: 0.4453 - learning_rate: 1.0000e-05
Epoch 13/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 34s 569ms/step - accuracy: 0.7714 - loss: 0.4624 - val_ac
curacy: 0.7695 - val_loss: 0.4575 - learning_rate: 1.0000e-05
Epoch 14/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 42s 577ms/step - accuracy: 0.8071 - loss: 0.4389 - val_ac
curacy: 0.7955 - val_loss: 0.4271 - learning_rate: 1.0000e-05
Epoch 15/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 568ms/step - accuracy: 0.7883 - loss: 0.4464 - val_ac
curacy: 0.7807 - val_loss: 0.4735 - learning_rate: 1.0000e-05
Epoch 16/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 574ms/step - accuracy: 0.8131 - loss: 0.4116 - val_ac
curacy: 0.7658 - val_loss: 0.4756 - learning_rate: 1.0000e-05
```

In [ ]:

```python
# Modeli oluştur
cnn_model = create_transfer_learning_model(input_shape, num_classes)

# İlk birkaç epoch'tan sonra dondurmayı kaldırın
for layer in cnn_model.layers[:15]:  # İlk 15 katmanı eğitime aç
    layer.trainable = True
```

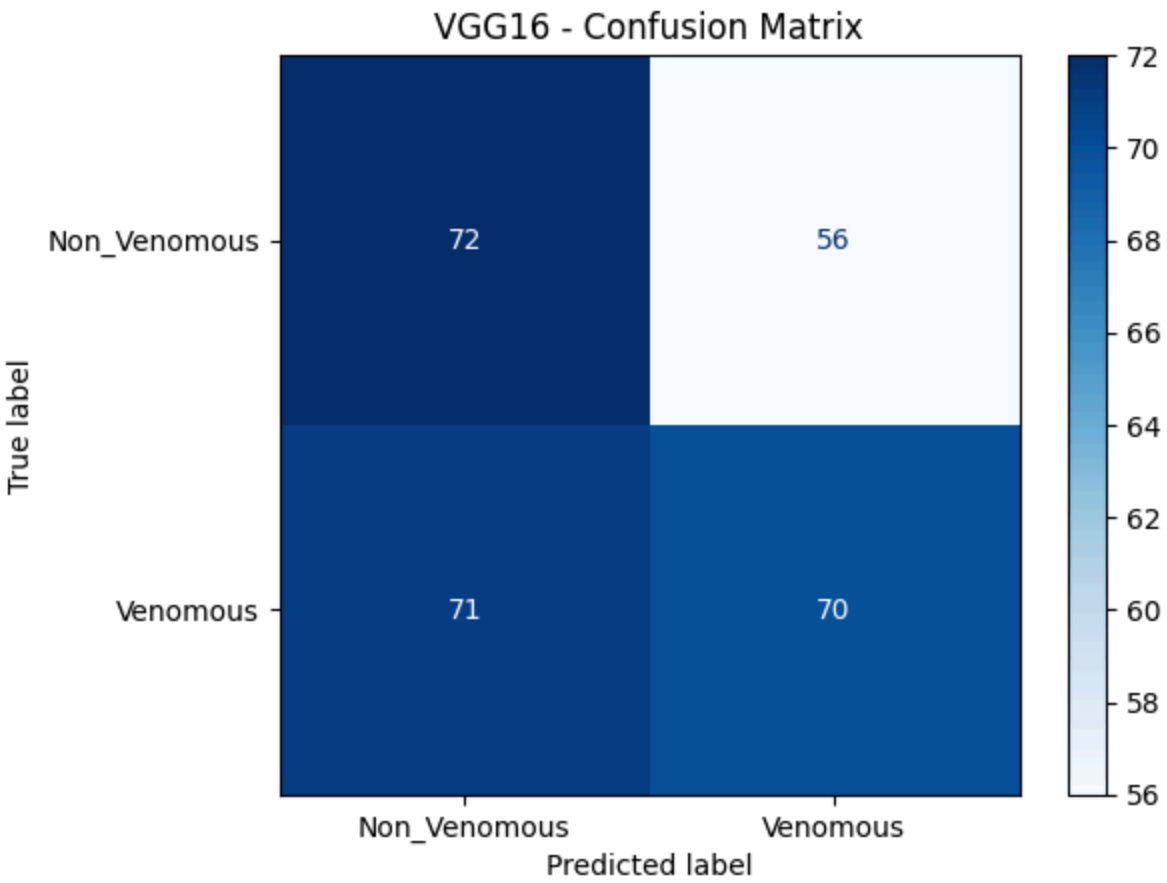
```
cnn_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),  # Daha düşük öğrenme h
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Model özeti
cnn_model.summary()

# Model eğitimi
history_vgg = cnn_model.fit(
    train_generator,
    epochs=50,  # Fine-tuning için birkaç epoch (ör. 10-20) ekleyebilirsiniz
    validation_data=test_generator,
    class_weight=class_weights,  # Sınıf ağırlıkları
    callbacks=[lr_scheduler_vgg,early_stop_vgg]  # Öğrenme oranını dinamik olarak a
)
```

Model Değerlendirme ve Confusion Matrix - VGG16

In [ ]:

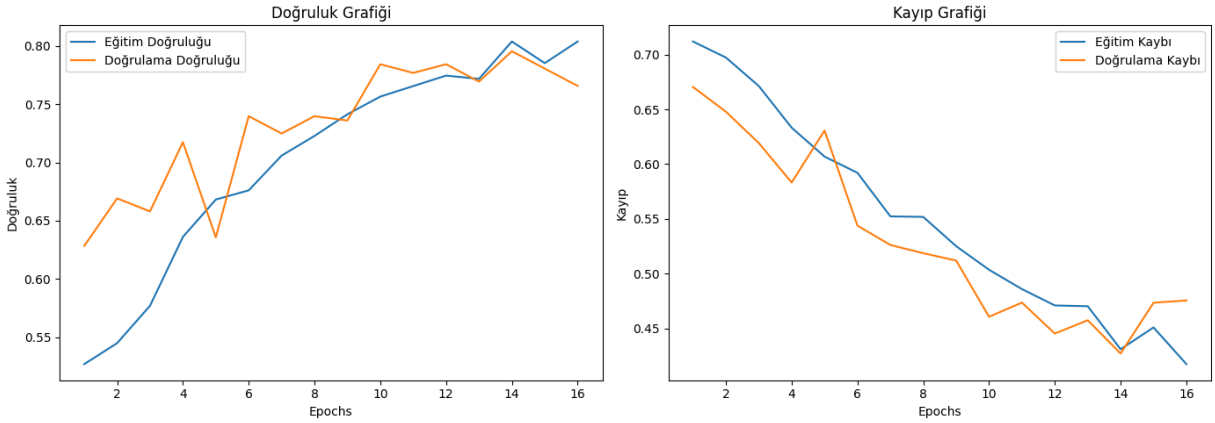
```
# Test verileri ile değerlendirme
test_loss_vgg, test_accuracy_vgg = cnn_model.evaluate(test_generator)
print(f"Test Loss: {test_loss_vgg}, Test Accuracy: {test_accuracy_vgg}")

# Confusion Matrix için tahminler
test_labels = test_generator.classes  # Test verisinin gerçek etiketleri
class_names = list(test_generator.class_indices.keys())  # Sınıf isimleri

vgg16_predictions = cnn_model.predict(test_generator)
vgg16_predicted_labels = np.round(vgg16_predictions).astype(int)  # Binary sınıflan

# Confusion Matrix Çizimi
cm_vgg = confusion_matrix(test_labels, vgg16_predicted_labels)
disp_vgg = ConfusionMatrixDisplay(confusion_matrix=cm_vgg, display_labels=class_nam
disp_vgg.plot(cmap=plt.cm.Blues)
plt.title("VGG16 - Confusion Matrix")
plt.show()
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 2s 169ms/step
```



In [ ]:
```
import matplotlib.pyplot as plt

# Eğitim ve doğrulama metriklerini alma
train_accuracy_vgg = history_vgg.history['accuracy']
val_accuracy_vgg = history_vgg.history['val_accuracy']
train_loss_vgg  = history_vgg.history['loss']
val_loss_vgg = history_vgg.history['val_loss']

# Epoch sayısını belirleme
epochs_vgg = range(1, len(train_accuracy_vgg) + 1)

# 1. Doğruluk grafiği
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.plot(epochs_vgg, train_accuracy_vgg, label='Eğitim Doğruluğu')
plt.plot(epochs_vgg, val_accuracy_vgg, label='Doğrulama Doğruluğu')
plt.title('Doğruluk Grafiği')
plt.xlabel('Epochs')
plt.ylabel('Doğruluk')
plt.legend()

# 2. Kayıp grafiği
plt.subplot(1, 2, 2)
plt.plot(epochs_vgg, train_loss_vgg, label='Eğitim Kaybı')
plt.plot(epochs_vgg, val_loss_vgg, label='Doğrulama Kaybı')
plt.title('Kayıp Grafiği')
plt.xlabel('Epochs')
```

```
plt.ylabel('Kayıp')
plt.legend()

plt.tight_layout()
plt.show()
```



MobilNetV2 modeli

In [ ]:
```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
import tensorflow as tf

# MobilNetV2 modeli
def create_mobilenetv2_model(input_shape, num_classes):
    base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=inp

    # Taban katmanlarını dondur
    for layer in base_model.layers:
        layer.trainable = False

    # Yeni katmanlar ekleyin
    x = base_model.output
    x = GlobalAveragePooling2D()(x)  # Havuzlama katmanı
    x = Dense(64, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.
    x = Dropout(0.4)(x)
    predictions = Dense(1, activation='sigmoid')(x)  # Binary sınıflandırma

    model = Model(inputs=base_model.input, outputs=predictions)
    return model

# Modeli oluştur
mobilenet_model = create_mobilenetv2_model(input_shape=(224, 224, 3), num_classes=1

# Modeli derle
mobilenet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/m
obilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

In [ ]:
```
# Model özeti
mobilenet_model.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #	Conne
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0	-
Conv1 (Conv2D)	(None, 112, 112, 32)	864	input
bn_Conv1 (BatchNormalization)	(None, 112, 112, 32)	128	Conv1
Conv1_relu (ReLU)	(None, 112, 112, 32)	0	bn_Co
expanded_conv_depthwise (DepthwiseConv2D)	(None, 112, 112, 32)	288	Conv1
expanded_conv_depthwise_… (BatchNormalization)	(None, 112, 112, 32)	128	expar
expanded_conv_depthwise_… (ReLU)	(None, 112, 112, 32)	0	expar
expanded_conv_project (Conv2D)	(None, 112, 112, 16)	512	expar
expanded_conv_project_BN (BatchNormalization)	(None, 112, 112, 16)	64	expar
block_1_expand (Conv2D)	(None, 112, 112, 96)	1,536	expar
block_1_expand_BN (BatchNormalization)	(None, 112, 112, 96)	384	block
block_1_expand_relu (ReLU)	(None, 112, 112, 96)	0	block
block_1_pad (ZeroPadding2D)	(None, 113, 113, 96)	0	block
block_1_depthwise (DepthwiseConv2D)	(None, 56, 56, 96)	864	block
block_1_depthwise_BN (BatchNormalization)	(None, 56, 56, 96)	384	block
block_1_depthwise_relu (ReLU)	(None, 56, 56, 96)	0	block
block_1_project (Conv2D)	(None, 56, 56, 24)	2,304	block
block_1_project_BN (BatchNormalization)	(None, 56, 56, 24)	96	block
block_2_expand (Conv2D)	(None, 56, 56, 144)	3,456	block
block_2_expand_BN	(None, 56, 56, 144)	576	block

(BatchNormalization)			
block_2_expand_relu (ReLU)	(None, 56, 56, 144)	0	block
block_2_depthwise (DepthwiseConv2D)	(None, 56, 56, 144)	1,296	block
block_2_depthwise_BN (BatchNormalization)	(None, 56, 56, 144)	576	block
block_2_depthwise_relu (ReLU)	(None, 56, 56, 144)	0	block
block_2_project (Conv2D)	(None, 56, 56, 24)	3,456	block
block_2_project_BN (BatchNormalization)	(None, 56, 56, 24)	96	block
block_2_add (Add)	(None, 56, 56, 24)	0	block block
block_3_expand (Conv2D)	(None, 56, 56, 144)	3,456	block
block_3_expand_BN (BatchNormalization)	(None, 56, 56, 144)	576	block
block_3_expand_relu (ReLU)	(None, 56, 56, 144)	0	block
block_3_pad (ZeroPadding2D)	(None, 57, 57, 144)	0	block
block_3_depthwise (DepthwiseConv2D)	(None, 28, 28, 144)	1,296	block
block_3_depthwise_BN (BatchNormalization)	(None, 28, 28, 144)	576	block
block_3_depthwise_relu (ReLU)	(None, 28, 28, 144)	0	block
block_3_project (Conv2D)	(None, 28, 28, 32)	4,608	block
block_3_project_BN (BatchNormalization)	(None, 28, 28, 32)	128	block
block_4_expand (Conv2D)	(None, 28, 28, 192)	6,144	block
block_4_expand_BN (BatchNormalization)	(None, 28, 28, 192)	768	block
block_4_expand_relu (ReLU)	(None, 28, 28, 192)	0	block
block_4_depthwise	(None, 28, 28, 192)	1,728	block

(DepthwiseConv2D)			
block_4_depthwise_BN (BatchNormalization)	(None, 28, 28, 192)	768	block
block_4_depthwise_relu (ReLU)	(None, 28, 28, 192)	0	block
block_4_project (Conv2D)	(None, 28, 28, 32)	6,144	block
block_4_project_BN (BatchNormalization)	(None, 28, 28, 32)	128	block
block_4_add (Add)	(None, 28, 28, 32)	0	block block
block_5_expand (Conv2D)	(None, 28, 28, 192)	6,144	block
block_5_expand_BN (BatchNormalization)	(None, 28, 28, 192)	768	block
block_5_expand_relu (ReLU)	(None, 28, 28, 192)	0	block
block_5_depthwise (DepthwiseConv2D)	(None, 28, 28, 192)	1,728	block
block_5_depthwise_BN (BatchNormalization)	(None, 28, 28, 192)	768	block
block_5_depthwise_relu (ReLU)	(None, 28, 28, 192)	0	block
block_5_project (Conv2D)	(None, 28, 28, 32)	6,144	block
block_5_project_BN (BatchNormalization)	(None, 28, 28, 32)	128	block
block_5_add (Add)	(None, 28, 28, 32)	0	block block
block_6_expand (Conv2D)	(None, 28, 28, 192)	6,144	block
block_6_expand_BN (BatchNormalization)	(None, 28, 28, 192)	768	block
block_6_expand_relu (ReLU)	(None, 28, 28, 192)	0	block
block_6_pad (ZeroPadding2D)	(None, 29, 29, 192)	0	block
block_6_depthwise (DepthwiseConv2D)	(None, 14, 14, 192)	1,728	block
block_6_depthwise_BN	(None, 14, 14, 192)	768	block

(BatchNormalization)			
block_6_depthwise_relu (ReLU)	(None, 14, 14, 192)	0	block
block_6_project (Conv2D)	(None, 14, 14, 64)	12,288	block
block_6_project_BN (BatchNormalization)	(None, 14, 14, 64)	256	block
block_7_expand (Conv2D)	(None, 14, 14, 384)	24,576	block
block_7_expand_BN (BatchNormalization)	(None, 14, 14, 384)	1,536	block
block_7_expand_relu (ReLU)	(None, 14, 14, 384)	0	block
block_7_depthwise (DepthwiseConv2D)	(None, 14, 14, 384)	3,456	block
block_7_depthwise_BN (BatchNormalization)	(None, 14, 14, 384)	1,536	block
block_7_depthwise_relu (ReLU)	(None, 14, 14, 384)	0	block
block_7_project (Conv2D)	(None, 14, 14, 64)	24,576	block
block_7_project_BN (BatchNormalization)	(None, 14, 14, 64)	256	block
block_7_add (Add)	(None, 14, 14, 64)	0	block block
block_8_expand (Conv2D)	(None, 14, 14, 384)	24,576	block
block_8_expand_BN (BatchNormalization)	(None, 14, 14, 384)	1,536	block
block_8_expand_relu (ReLU)	(None, 14, 14, 384)	0	block
block_8_depthwise (DepthwiseConv2D)	(None, 14, 14, 384)	3,456	block
block_8_depthwise_BN (BatchNormalization)	(None, 14, 14, 384)	1,536	block
block_8_depthwise_relu (ReLU)	(None, 14, 14, 384)	0	block
block_8_project (Conv2D)	(None, 14, 14, 64)	24,576	block
block_8_project_BN (BatchNormalization)	(None, 14, 14, 64)	256	block

block_8_add (Add)	(None, 14, 14, 64)	0	block block
block_9_expand (Conv2D)	(None, 14, 14, 384)	24,576	block
block_9_expand_BN (BatchNormalization)	(None, 14, 14, 384)	1,536	block
block_9_expand_relu (ReLU)	(None, 14, 14, 384)	0	block
block_9_depthwise (DepthwiseConv2D)	(None, 14, 14, 384)	3,456	block
block_9_depthwise_BN (BatchNormalization)	(None, 14, 14, 384)	1,536	block
block_9_depthwise_relu (ReLU)	(None, 14, 14, 384)	0	block
block_9_project (Conv2D)	(None, 14, 14, 64)	24,576	block
block_9_project_BN (BatchNormalization)	(None, 14, 14, 64)	256	block
block_9_add (Add)	(None, 14, 14, 64)	0	block block
block_10_expand (Conv2D)	(None, 14, 14, 384)	24,576	block
block_10_expand_BN (BatchNormalization)	(None, 14, 14, 384)	1,536	block
block_10_expand_relu (ReLU)	(None, 14, 14, 384)	0	block
block_10_depthwise (DepthwiseConv2D)	(None, 14, 14, 384)	3,456	block
block_10_depthwise_BN (BatchNormalization)	(None, 14, 14, 384)	1,536	block
block_10_depthwise_relu (ReLU)	(None, 14, 14, 384)	0	block
block_10_project (Conv2D)	(None, 14, 14, 96)	36,864	block
block_10_project_BN (BatchNormalization)	(None, 14, 14, 96)	384	block
block_11_expand (Conv2D)	(None, 14, 14, 576)	55,296	block
block_11_expand_BN (BatchNormalization)	(None, 14, 14, 576)	2,304	block

block_11_expand_relu (ReLU)	(None, 14, 14, 576)	0	block
block_11_depthwise (DepthwiseConv2D)	(None, 14, 14, 576)	5,184	block
block_11_depthwise_BN (BatchNormalization)	(None, 14, 14, 576)	2,304	block
block_11_depthwise_relu (ReLU)	(None, 14, 14, 576)	0	block
block_11_project (Conv2D)	(None, 14, 14, 96)	55,296	block
block_11_project_BN (BatchNormalization)	(None, 14, 14, 96)	384	block
block_11_add (Add)	(None, 14, 14, 96)	0	block block
block_12_expand (Conv2D)	(None, 14, 14, 576)	55,296	block
block_12_expand_BN (BatchNormalization)	(None, 14, 14, 576)	2,304	block
block_12_expand_relu (ReLU)	(None, 14, 14, 576)	0	block
block_12_depthwise (DepthwiseConv2D)	(None, 14, 14, 576)	5,184	block
block_12_depthwise_BN (BatchNormalization)	(None, 14, 14, 576)	2,304	block
block_12_depthwise_relu (ReLU)	(None, 14, 14, 576)	0	block
block_12_project (Conv2D)	(None, 14, 14, 96)	55,296	block
block_12_project_BN (BatchNormalization)	(None, 14, 14, 96)	384	block
block_12_add (Add)	(None, 14, 14, 96)	0	block block
block_13_expand (Conv2D)	(None, 14, 14, 576)	55,296	block
block_13_expand_BN (BatchNormalization)	(None, 14, 14, 576)	2,304	block
block_13_expand_relu (ReLU)	(None, 14, 14, 576)	0	block
block_13_pad (ZeroPadding2D)	(None, 15, 15, 576)	0	block

block_13_depthwise (DepthwiseConv2D)	(None, 7, 7, 576)	5,184	block
block_13_depthwise_BN (BatchNormalization)	(None, 7, 7, 576)	2,304	block
block_13_depthwise_relu (ReLU)	(None, 7, 7, 576)	0	block
block_13_project (Conv2D)	(None, 7, 7, 160)	92,160	block
block_13_project_BN (BatchNormalization)	(None, 7, 7, 160)	640	block
block_14_expand (Conv2D)	(None, 7, 7, 960)	153,600	block
block_14_expand_BN (BatchNormalization)	(None, 7, 7, 960)	3,840	block
block_14_expand_relu (ReLU)	(None, 7, 7, 960)	0	block
block_14_depthwise (DepthwiseConv2D)	(None, 7, 7, 960)	8,640	block
block_14_depthwise_BN (BatchNormalization)	(None, 7, 7, 960)	3,840	block
block_14_depthwise_relu (ReLU)	(None, 7, 7, 960)	0	block
block_14_project (Conv2D)	(None, 7, 7, 160)	153,600	block
block_14_project_BN (BatchNormalization)	(None, 7, 7, 160)	640	block
block_14_add (Add)	(None, 7, 7, 160)	0	block block
block_15_expand (Conv2D)	(None, 7, 7, 960)	153,600	block
block_15_expand_BN (BatchNormalization)	(None, 7, 7, 960)	3,840	block
block_15_expand_relu (ReLU)	(None, 7, 7, 960)	0	block
block_15_depthwise (DepthwiseConv2D)	(None, 7, 7, 960)	8,640	block
block_15_depthwise_BN (BatchNormalization)	(None, 7, 7, 960)	3,840	block
block_15_depthwise_relu (ReLU)	(None, 7, 7, 960)	0	block

block_15_project (Conv2D)	(None, 7, 7, 160)	153,600	block
block_15_project_BN (BatchNormalization)	(None, 7, 7, 160)	640	block
block_15_add (Add)	(None, 7, 7, 160)	0	block block
block_16_expand (Conv2D)	(None, 7, 7, 960)	153,600	block
block_16_expand_BN (BatchNormalization)	(None, 7, 7, 960)	3,840	block
block_16_expand_relu (ReLU)	(None, 7, 7, 960)	0	block
block_16_depthwise (DepthwiseConv2D)	(None, 7, 7, 960)	8,640	block
block_16_depthwise_BN (BatchNormalization)	(None, 7, 7, 960)	3,840	block
block_16_depthwise_relu (ReLU)	(None, 7, 7, 960)	0	block
block_16_project (Conv2D)	(None, 7, 7, 320)	307,200	block
block_16_project_BN (BatchNormalization)	(None, 7, 7, 320)	1,280	block
Conv_1 (Conv2D)	(None, 7, 7, 1280)	409,600	block
Conv_1_bn (BatchNormalization)	(None, 7, 7, 1280)	5,120	Conv_
out_relu (ReLU)	(None, 7, 7, 1280)	0	Conv_
global_average_pooling2d… (GlobalAveragePooling2D)	(None, 1280)	0	out_r
dense_4 (Dense)	(None, 64)	81,984	globa
dropout_2 (Dropout)	(None, 64)	0	dense
dense_5 (Dense)	(None, 1)	65	dropc

Total params: 2,340,033 (8.93 MB)

Trainable params: 82,049 (320.50 KB)

Non-trainable params: 2,257,984 (8.61 MB)

In [ ]:
```
# Model özeti
mobilenet_model.summary()

# Sınıf ağırlıklarını hesaplama
```

```
class_weights_array = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_generator.classes),
    y=train_generator.classes
)
class_weights = {i: weight for i, weight in enumerate(class_weights_array)}

# Callbacks tanımlama
lr_scheduler_mobile = ReduceLROnPlateau(
    monitor='val_loss',   # Hangi metriği takip edecek
    factor=0.5,           # Öğrenme hızını yarıya indir
    patience=3,           # Kaç epoch bekleyecek
    min_lr=1e-6           # Minimum öğrenme hızı
)

early_stop_mobile = EarlyStopping(
    monitor='val_loss',
    patience=7,  # Bekleme süresini 10 epoch'a çıkardık
    min_delta=0.001,
    restore_best_weights=True
)




# Model eğitimi
history_mobile = mobilenet_model.fit(
    train_generator,
    epochs=50,
    validation_data=test_generator,
    class_weight=class_weights,
    callbacks=[lr_scheduler_mobile, early_stop_mobile]  # ReduceLROnPlateau ve Earl
)
```

```
Epoch 1/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 43s 533ms/step - accuracy: 0.5661 - loss: 1.9807 - val_ac
curacy: 0.6952 - val_loss: 1.7510 - learning_rate: 1.0000e-04
Epoch 2/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 26s 376ms/step - accuracy: 0.6560 - loss: 1.7671 - val_ac
curacy: 0.7361 - val_loss: 1.6333 - learning_rate: 1.0000e-04
Epoch 3/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 351ms/step - accuracy: 0.6804 - loss: 1.6757 - val_ac
curacy: 0.7584 - val_loss: 1.5473 - learning_rate: 1.0000e-04
Epoch 4/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 42s 371ms/step - accuracy: 0.7343 - loss: 1.5669 - val_ac
curacy: 0.8030 - val_loss: 1.4706 - learning_rate: 1.0000e-04
Epoch 5/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 370ms/step - accuracy: 0.7349 - loss: 1.5141 - val_ac
curacy: 0.8141 - val_loss: 1.4057 - learning_rate: 1.0000e-04
Epoch 6/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 345ms/step - accuracy: 0.7625 - loss: 1.4364 - val_ac
curacy: 0.8141 - val_loss: 1.3547 - learning_rate: 1.0000e-04
Epoch 7/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 343ms/step - accuracy: 0.7716 - loss: 1.3771 - val_ac
curacy: 0.8327 - val_loss: 1.2947 - learning_rate: 1.0000e-04
Epoch 8/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 367ms/step - accuracy: 0.7794 - loss: 1.3309 - val_ac
curacy: 0.8216 - val_loss: 1.2480 - learning_rate: 1.0000e-04
Epoch 9/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 351ms/step - accuracy: 0.7756 - loss: 1.2743 - val_ac
curacy: 0.8327 - val_loss: 1.2041 - learning_rate: 1.0000e-04
Epoch 10/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 42s 369ms/step - accuracy: 0.7900 - loss: 1.2225 - val_ac
curacy: 0.8401 - val_loss: 1.1667 - learning_rate: 1.0000e-04
Epoch 11/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 347ms/step - accuracy: 0.8028 - loss: 1.1952 - val_ac
curacy: 0.8290 - val_loss: 1.1302 - learning_rate: 1.0000e-04
Epoch 12/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 348ms/step - accuracy: 0.8091 - loss: 1.1316 - val_ac
curacy: 0.8216 - val_loss: 1.0994 - learning_rate: 1.0000e-04
Epoch 13/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 354ms/step - accuracy: 0.8139 - loss: 1.1268 - val_ac
curacy: 0.8401 - val_loss: 1.0633 - learning_rate: 1.0000e-04
Epoch 14/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 348ms/step - accuracy: 0.8309 - loss: 1.0691 - val_ac
curacy: 0.8290 - val_loss: 1.0389 - learning_rate: 1.0000e-04
Epoch 15/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 350ms/step - accuracy: 0.8143 - loss: 1.0538 - val_ac
curacy: 0.8401 - val_loss: 1.0049 - learning_rate: 1.0000e-04
Epoch 16/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 347ms/step - accuracy: 0.8029 - loss: 1.0341 - val_ac
curacy: 0.8439 - val_loss: 0.9845 - learning_rate: 1.0000e-04
Epoch 17/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 363ms/step - accuracy: 0.8258 - loss: 0.9955 - val_ac
curacy: 0.8364 - val_loss: 0.9584 - learning_rate: 1.0000e-04
Epoch 18/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 374ms/step - accuracy: 0.7929 - loss: 1.0012 - val_ac
curacy: 0.8364 - val_loss: 0.9422 - learning_rate: 1.0000e-04
Epoch 19/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 350ms/step - accuracy: 0.8197 - loss: 0.9500 - val_ac
```

```
curacy: 0.8253 - val_loss: 0.9187 - learning_rate: 1.0000e-04
Epoch 20/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 42s 376ms/step - accuracy: 0.8320 - loss: 0.9192 - val_ac
curacy: 0.8401 - val_loss: 0.9006 - learning_rate: 1.0000e-04
Epoch 21/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 381ms/step - accuracy: 0.8389 - loss: 0.8999 - val_ac
curacy: 0.8401 - val_loss: 0.8744 - learning_rate: 1.0000e-04
Epoch 22/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 353ms/step - accuracy: 0.8315 - loss: 0.8850 - val_ac
curacy: 0.8364 - val_loss: 0.8655 - learning_rate: 1.0000e-04
Epoch 23/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 355ms/step - accuracy: 0.8312 - loss: 0.8698 - val_ac
curacy: 0.8364 - val_loss: 0.8458 - learning_rate: 1.0000e-04
Epoch 24/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 371ms/step - accuracy: 0.8507 - loss: 0.8266 - val_ac
curacy: 0.8439 - val_loss: 0.8248 - learning_rate: 1.0000e-04
Epoch 25/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 351ms/step - accuracy: 0.8575 - loss: 0.8117 - val_ac
curacy: 0.8476 - val_loss: 0.8080 - learning_rate: 1.0000e-04
Epoch 26/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 377ms/step - accuracy: 0.8557 - loss: 0.7947 - val_ac
curacy: 0.8476 - val_loss: 0.7941 - learning_rate: 1.0000e-04
Epoch 27/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 361ms/step - accuracy: 0.8554 - loss: 0.7797 - val_ac
curacy: 0.8327 - val_loss: 0.7810 - learning_rate: 1.0000e-04
Epoch 28/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 42s 374ms/step - accuracy: 0.8462 - loss: 0.7820 - val_ac
curacy: 0.8439 - val_loss: 0.7680 - learning_rate: 1.0000e-04
Epoch 29/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 348ms/step - accuracy: 0.8557 - loss: 0.7524 - val_ac
curacy: 0.8401 - val_loss: 0.7564 - learning_rate: 1.0000e-04
Epoch 30/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 373ms/step - accuracy: 0.8592 - loss: 0.7413 - val_ac
curacy: 0.8476 - val_loss: 0.7441 - learning_rate: 1.0000e-04
Epoch 31/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 358ms/step - accuracy: 0.8496 - loss: 0.7297 - val_ac
curacy: 0.8439 - val_loss: 0.7336 - learning_rate: 1.0000e-04
Epoch 32/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 375ms/step - accuracy: 0.8492 - loss: 0.7318 - val_ac
curacy: 0.8439 - val_loss: 0.7200 - learning_rate: 1.0000e-04
Epoch 33/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 357ms/step - accuracy: 0.8503 - loss: 0.7091 - val_ac
curacy: 0.8513 - val_loss: 0.7122 - learning_rate: 1.0000e-04
Epoch 34/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 44s 421ms/step - accuracy: 0.8777 - loss: 0.6686 - val_ac
curacy: 0.8513 - val_loss: 0.7055 - learning_rate: 1.0000e-04
Epoch 35/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 384ms/step - accuracy: 0.8693 - loss: 0.6695 - val_ac
curacy: 0.8401 - val_loss: 0.6891 - learning_rate: 1.0000e-04
Epoch 36/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 365ms/step - accuracy: 0.8575 - loss: 0.6825 - val_ac
curacy: 0.8476 - val_loss: 0.6776 - learning_rate: 1.0000e-04
Epoch 37/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 352ms/step - accuracy: 0.8596 - loss: 0.6466 - val_ac
curacy: 0.8513 - val_loss: 0.6703 - learning_rate: 1.0000e-04
Epoch 38/50
```

```
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 362ms/step - accuracy: 0.8647 - loss: 0.6363 - val_ac
curacy: 0.8513 - val_loss: 0.6624 - learning_rate: 1.0000e-04
Epoch 39/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 354ms/step - accuracy: 0.8653 - loss: 0.6441 - val_ac
curacy: 0.8699 - val_loss: 0.6545 - learning_rate: 1.0000e-04
Epoch 40/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 365ms/step - accuracy: 0.8551 - loss: 0.6333 - val_ac
curacy: 0.8699 - val_loss: 0.6463 - learning_rate: 1.0000e-04
Epoch 41/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 373ms/step - accuracy: 0.8664 - loss: 0.6140 - val_ac
curacy: 0.8550 - val_loss: 0.6405 - learning_rate: 1.0000e-04
Epoch 42/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 360ms/step - accuracy: 0.8634 - loss: 0.6135 - val_ac
curacy: 0.8699 - val_loss: 0.6279 - learning_rate: 1.0000e-04
Epoch 43/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 351ms/step - accuracy: 0.8805 - loss: 0.5832 - val_ac
curacy: 0.8476 - val_loss: 0.6195 - learning_rate: 1.0000e-04
Epoch 44/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 362ms/step - accuracy: 0.8605 - loss: 0.5966 - val_ac
curacy: 0.8625 - val_loss: 0.6108 - learning_rate: 1.0000e-04
Epoch 45/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 25s 401ms/step - accuracy: 0.8784 - loss: 0.5827 - val_ac
curacy: 0.8662 - val_loss: 0.6052 - learning_rate: 1.0000e-04
Epoch 46/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 24s 377ms/step - accuracy: 0.8730 - loss: 0.5704 - val_ac
curacy: 0.8625 - val_loss: 0.6004 - learning_rate: 1.0000e-04
Epoch 47/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 22s 349ms/step - accuracy: 0.8860 - loss: 0.5692 - val_ac
curacy: 0.8662 - val_loss: 0.5915 - learning_rate: 1.0000e-04
Epoch 48/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 345ms/step - accuracy: 0.8936 - loss: 0.5528 - val_ac
curacy: 0.8662 - val_loss: 0.5867 - learning_rate: 1.0000e-04
Epoch 49/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 353ms/step - accuracy: 0.8804 - loss: 0.5378 - val_ac
curacy: 0.8625 - val_loss: 0.5818 - learning_rate: 1.0000e-04
Epoch 50/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 23s 350ms/step - accuracy: 0.8859 - loss: 0.5360 - val_ac
curacy: 0.8662 - val_loss: 0.5794 - learning_rate: 1.0000e-04
```

In [ ]:
```
# Modeli kaydetme
mobilenet_model.save("snake_classification_model.keras")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.
saving.save_model(model)`. This file format is considered legacy. We recommend using
instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.savin
g.save_model(model, 'my_model.keras')`.
```

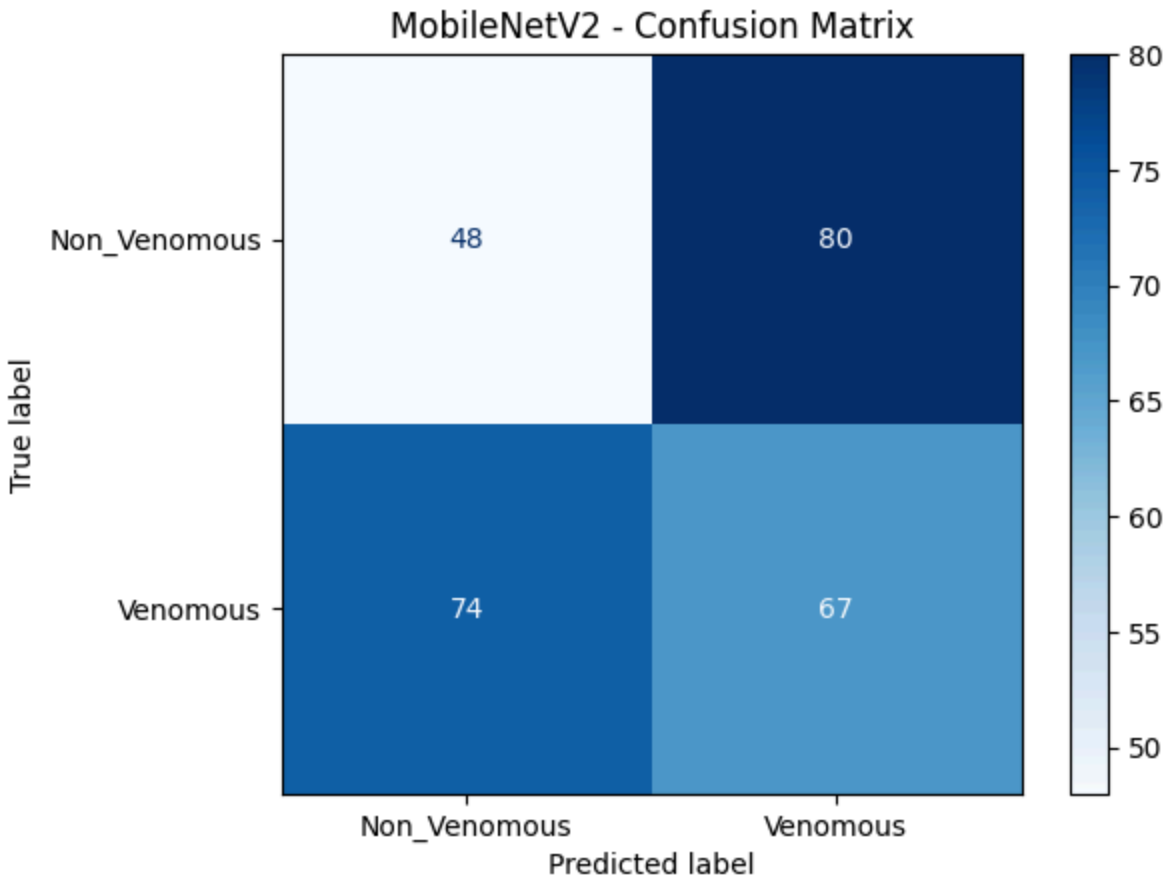
In [ ]:
```
# Test verileri ile değerlendirme
test_loss, test_accuracy = mobilenet_model.evaluate(test_generator)
print(f"Test Loss: {test_loss}, Test Accuracy: {test_accuracy}")
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 1s 64ms/step - accuracy: 0.8924 - loss: 0.5477
Test Loss: 0.5793561339378357, Test Accuracy: 0.8661710023880005
```

In [ ]:
```
# Confusion Matrix için tahminler
test_labels = test_generator.classes  # Test verisinin gerçek etiketleri
class_names = list(test_generator.class_indices.keys())  # Sınıf isimleri
```

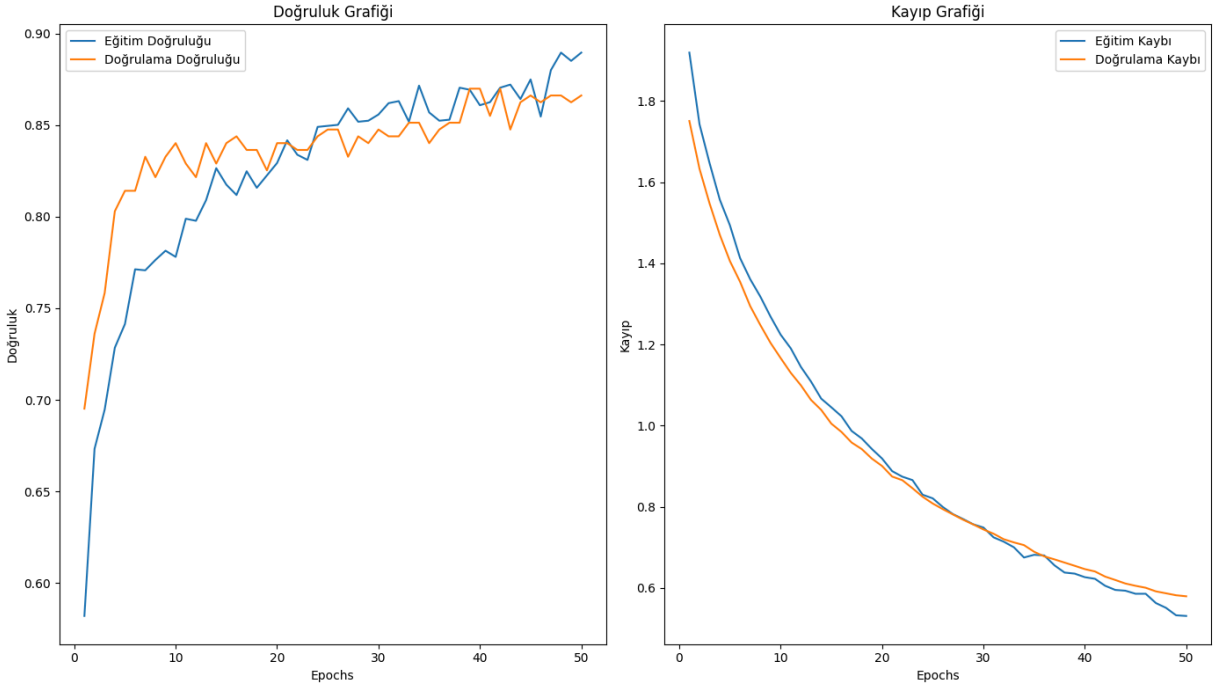
```
mobilenet_predictions = mobilenet_model.predict(test_generator)
mobilenet_predicted_labels = np.round(mobilenet_predictions).astype(int)  # Binary

# Confusion Matrix Çizimi
cm_mobile = confusion_matrix(test_labels, mobilenet_predicted_labels)
disp_mobile = ConfusionMatrixDisplay(confusion_matrix=cm_mobile, display_labels=cla
disp_mobile.plot(cmap=plt.cm.Blues)
plt.title("MobileNetV2 - Confusion Matrix")
plt.show()
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 7s 509ms/step
```



In [ ]:
```
import matplotlib.pyplot as plt

# Eğitim ve doğrulama metriklerini alma
train_acc_mobile = history_mobile.history['accuracy']
val_acc_mobile = history_mobile.history['val_accuracy']
train_loss_mobile = history_mobile.history['loss']
val_loss_mobile = history_mobile.history['val_loss']

# Epoch sayısını belirleme
epochs_mobile = range(1, len(train_acc_mobile) + 1)

# 1. Doğruluk grafiği
plt.figure(figsize=(14, 8))

plt.subplot(1, 2, 1)
plt.plot(epochs_mobile, train_acc_mobile, label='Eğitim Doğruluğu')
```

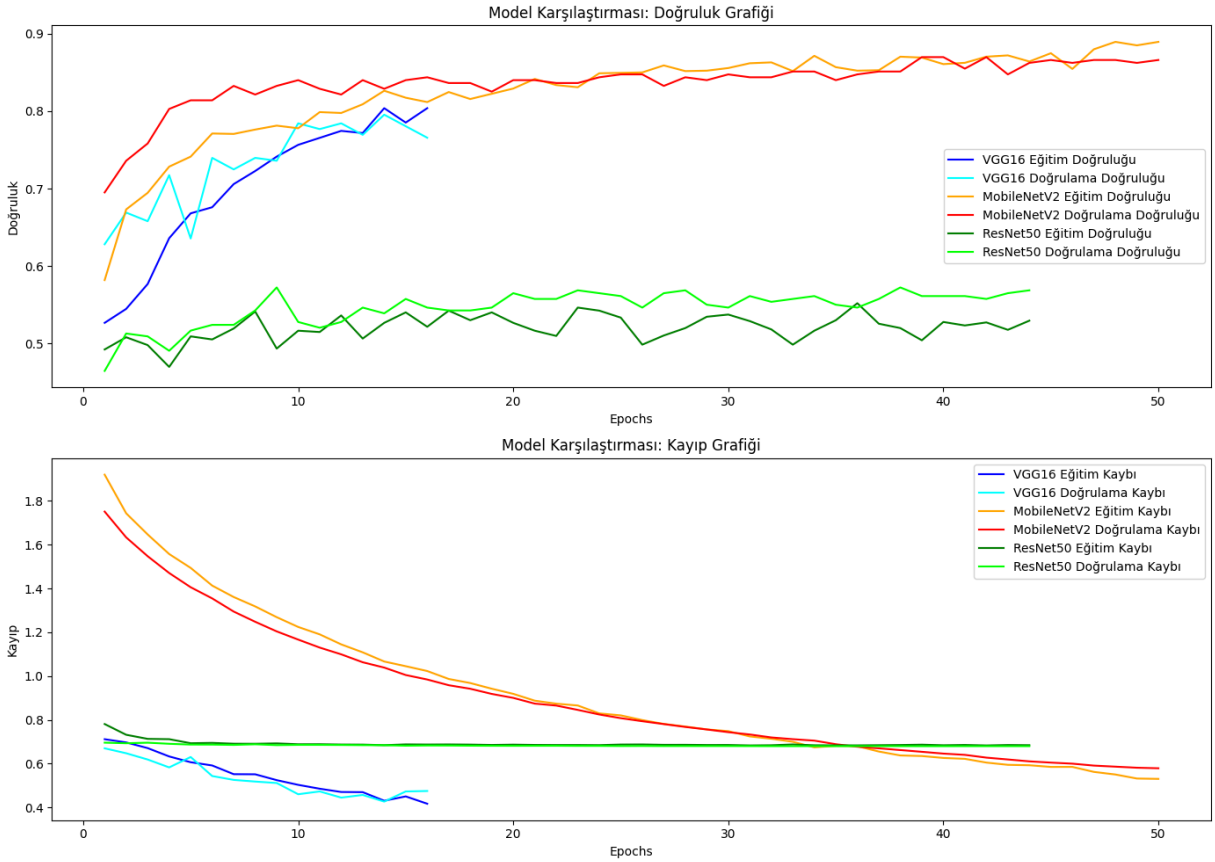
```
plt.plot(epochs_mobile, val_acc_mobile, label='Doğrulama Doğruluğu')
plt.title('Doğruluk Grafiği')
plt.xlabel('Epochs')
plt.ylabel('Doğruluk')
plt.legend()

# 2. Kayıp grafiği
plt.subplot(1, 2, 2)
plt.plot(epochs_mobile, train_loss_mobile, label='Eğitim Kaybı')
plt.plot(epochs_mobile, val_loss_mobile, label='Doğrulama Kaybı')
plt.title('Kayıp Grafiği')
plt.xlabel('Epochs')
plt.ylabel('Kayıp')
plt.legend()

plt.tight_layout()
plt.show()
```



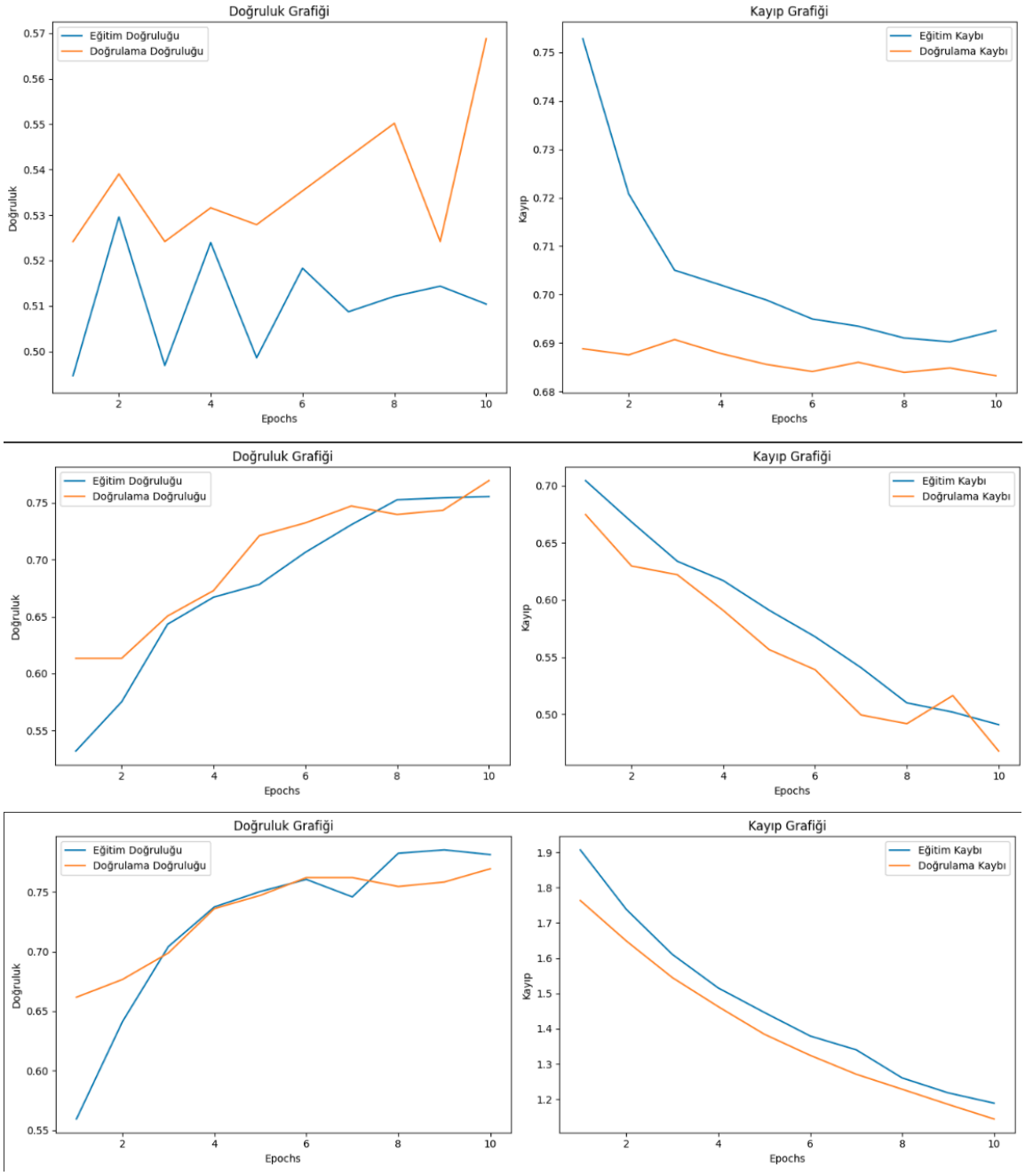
Model Karşılaştırması (VGG16, MobileNetV2, ResNet50)

In [ ]:
```
# Epoch sayısı
epochs_vgg  = range(1, len(train_accuracy_vgg) + 1)
epochs_mobile = range(1, len(train_acc_mobile) + 1)
epochs_res = range(1, len(train_acc_res) + 1)

# Grafik çizimi
plt.figure(figsize=(14, 10))

# 1. Doğruluk Karşılaştırması
plt.subplot(2, 1, 1)
plt.plot(epochs_vgg, train_accuracy_vgg, label='VGG16 Eğitim Doğruluğu', color='blu
plt.plot(epochs_vgg, val_accuracy_vgg, label='VGG16 Doğrulama Doğruluğu', color='cy
```

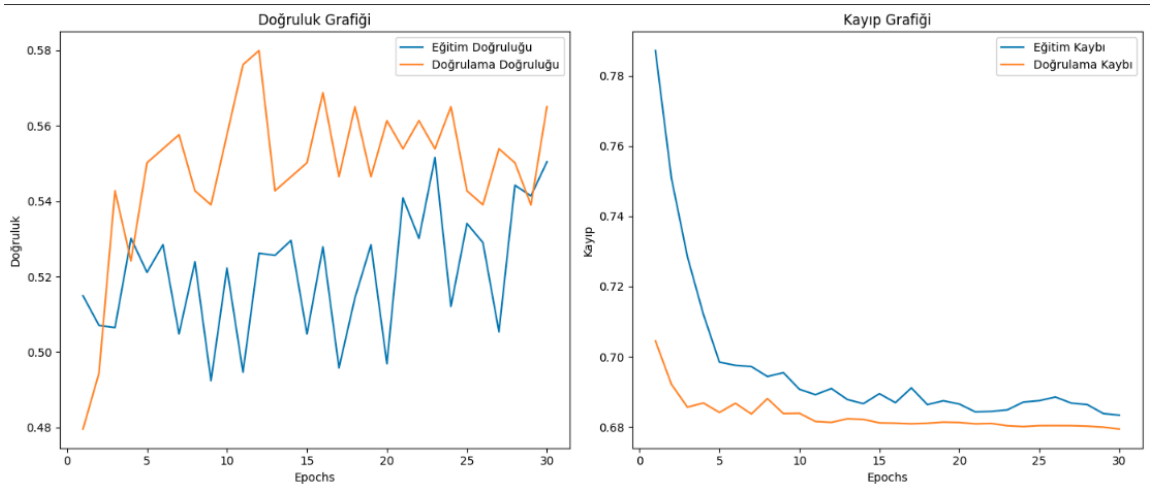
```
plt.plot(epochs_mobile, train_acc_mobile, label='MobileNetV2 Eğitim Doğruluğu', col
plt.plot(epochs_mobile, val_acc_mobile, label='MobileNetV2 Doğrulama Doğruluğu', co
plt.plot(epochs_res, train_acc_res, label='ResNet50 Eğitim Doğruluğu', color='green
plt.plot(epochs_res, val_acc_res, label='ResNet50 Doğrulama Doğruluğu', color='lime
plt.title("Model Karşılaştırması: Doğruluk Grafiği")
plt.xlabel("Epochs")
plt.ylabel("Doğruluk")
plt.legend()

# 2. Kayıp Karşılaştırması
plt.subplot(2, 1, 2)
plt.plot(epochs_vgg, train_loss_vgg, label='VGG16 Eğitim Kaybı', color='blue')
plt.plot(epochs_vgg, val_loss_vgg, label='VGG16 Doğrulama Kaybı', color='cyan')
plt.plot(epochs_mobile, train_loss_mobile, label='MobileNetV2 Eğitim Kaybı', color=
plt.plot(epochs_mobile, val_loss_mobile, label='MobileNetV2 Doğrulama Kaybı', color
plt.plot(epochs_res, train_loss_res, label='ResNet50 Eğitim Kaybı', color='green')
plt.plot(epochs_res, val_loss_res, label='ResNet50 Doğrulama Kaybı', color='lime')
plt.title("Model Karşılaştırması: Kayıp Grafiği")
plt.xlabel("Epochs")
plt.ylabel("Kayıp")
plt.legend()

plt.tight_layout()
plt.show()
```



10 Epoch performans karşılaştırılması

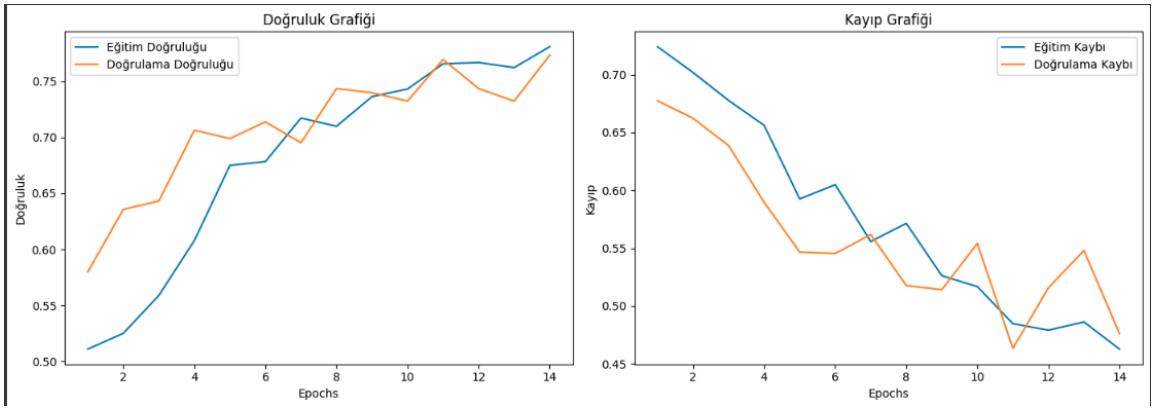


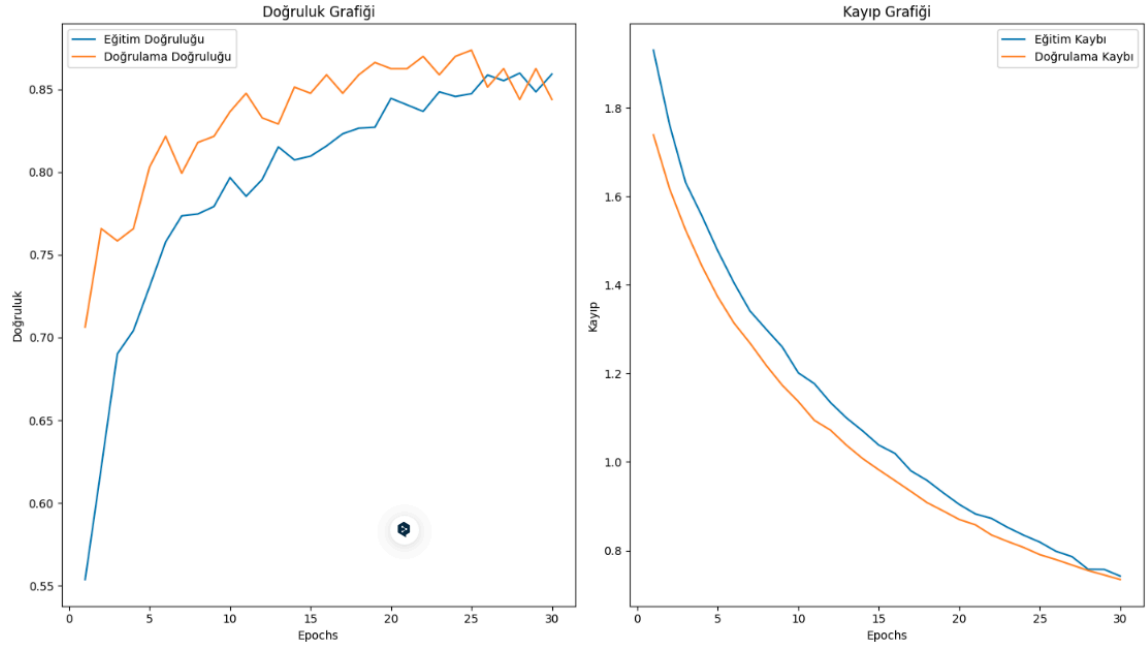
30 epoch model karşılaştırması



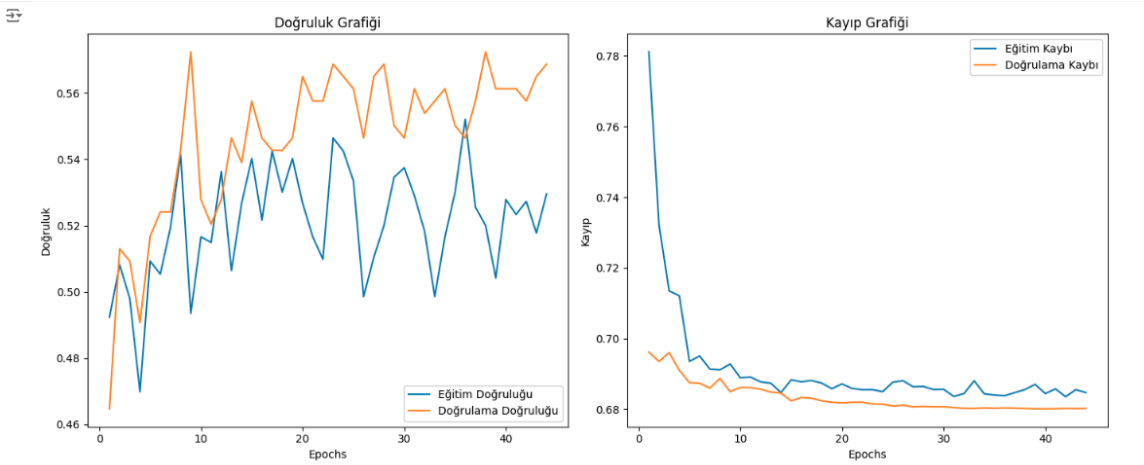

```
# Model eğitimi
vgg16_history = cnn_model.fit(
    train_generator,
    epochs=30,  # Fine-tuning için birkaç epoch (ör. 10-20) ekleyebilirsiniz
    validation_data=test_generator,
    class_weight=class_weights,  # Sınıf ağırlıkları
    callbacks=[lr_scheduler_VGG16,VGG16_early_stop]  # Öğrenme oranını dinamik olarak ayarlamak için
)
```

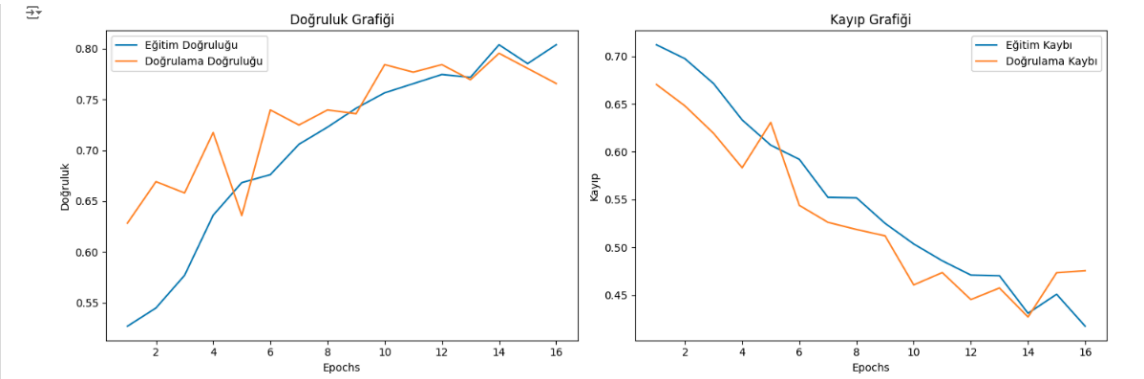
```
Epoch 1/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 29s 384ms/step - accuracy: 0.5101 - loss: 0.7303 - val_accuracy: 0.5799 - val_loss: 0.6774 - learning_rate: 1.0000e-05
Epoch 2/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 21s 324ms/step - accuracy: 0.5434 - loss: 0.6940 - val_accuracy: 0.6357 - val_loss: 0.6623 - learning_rate: 1.0000e-05
Epoch 3/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 21s 328ms/step - accuracy: 0.5406 - loss: 0.6882 - val_accuracy: 0.6431 - val_loss: 0.6386 - learning_rate: 1.0000e-05
Epoch 4/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 21s 321ms/step - accuracy: 0.6052 - loss: 0.6553 - val_accuracy: 0.7063 - val_loss: 0.5896 - learning_rate: 1.0000e-05
Epoch 5/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 20s 320ms/step - accuracy: 0.6617 - loss: 0.6016 - val_accuracy: 0.6989 - val_loss: 0.5465 - learning_rate: 1.0000e-05
Epoch 6/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 21s 324ms/step - accuracy: 0.6894 - loss: 0.6046 - val_accuracy: 0.7138 - val_loss: 0.5454 - learning_rate: 1.0000e-05
Epoch 7/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 21s 321ms/step - accuracy: 0.7103 - loss: 0.5601 - val_accuracy: 0.6952 - val_loss: 0.5617 - learning_rate: 1.0000e-05
Epoch 8/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 21s 322ms/step - accuracy: 0.7263 - loss: 0.5636 - val_accuracy: 0.7435 - val_loss: 0.5176 - learning_rate: 1.0000e-05
Epoch 9/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 20s 320ms/step - accuracy: 0.7379 - loss: 0.5347 - val_accuracy: 0.7398 - val_loss: 0.5140 - learning_rate: 1.0000e-05
Epoch 10/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 20s 319ms/step - accuracy: 0.7509 - loss: 0.4941 - val_accuracy: 0.7323 - val_loss: 0.5540 - learning_rate: 1.0000e-05
Epoch 11/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 20s 319ms/step - accuracy: 0.7622 - loss: 0.4815 - val_accuracy: 0.7695 - val_loss: 0.4632 - learning_rate: 1.0000e-05
Epoch 12/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 20s 319ms/step - accuracy: 0.7751 - loss: 0.4745 - val_accuracy: 0.7435 - val_loss: 0.5156 - learning_rate: 1.0000e-05
Epoch 13/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 21s 327ms/step - accuracy: 0.7671 - loss: 0.4711 - val_accuracy: 0.7323 - val_loss: 0.5479 - learning_rate: 1.0000e-05
Epoch 14/30
56/56 ━━━━━━━━━━━━━━━━━━━━ 21s 323ms/step - accuracy: 0.7583 - loss: 0.4818 - val_accuracy: 0.7732 - val_loss: 0.4760 - learning_rate: 1.0000e-05
```





50 epoch model karşılaştırması






```
Epoch 12/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 40s 573ms/step - accuracy: 0.7966 - loss: 0.4539 - val_accuracy: 0.7844 - val_loss: 0.4453 - learning_rate: 1.0000e-05
Epoch 13/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 34s 569ms/step - accuracy: 0.7714 - loss: 0.4624 - val_accuracy: 0.7695 - val_loss: 0.4575 - learning_rate: 1.0000e-05
Epoch 14/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 42s 577ms/step - accuracy: 0.8071 - loss: 0.4389 - val_accuracy: 0.7955 - val_loss: 0.4271 - learning_rate: 1.0000e-05
Epoch 15/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 568ms/step - accuracy: 0.7883 - loss: 0.4464 - val_accuracy: 0.7807 - val_loss: 0.4735 - learning_rate: 1.0000e-05
Epoch 16/50
56/56 ━━━━━━━━━━━━━━━━━━━━ 41s 574ms/step - accuracy: 0.8131 - loss: 0.4116 - val_accuracy: 0.7658 - val_loss: 0.4756 - learning_rate: 1.0000e-05
```

```
[15] # Test verileri ile değerlendirme
     test_loss_vgg, test_accuracy_vgg = cnn_model.evaluate(test_generator)
     print(f"Test Loss: {test_loss_vgg}, Test Accuracy: {test_accuracy_vgg}")
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 1s 125ms/step - accuracy: 0.7963 - loss: 0.4247
Test Loss: 0.4271198511123657, Test Accuracy: 0.7955390214920044
```

Genel Karşılaştırma Tablosu

Model	Epochs	Optimizer	Test Accuracy (%)
ResNet	10	Adam	56
VGG16	10	Adam	76
MobileNet	10	Adam	77
ResNet	30	Adam	56
VGG16	30	Adam	77
MobileNet	30	Adam	84
ResNet	50	Adam	57
VGG16	50	Adam	79
MobileNet	**50**	**Adam**	**87** 🔥